School of Computer Science and Artificial Intelligence

Lab Assignment # 22.5

Program : B. Tech (CSE)

Specialization :AIML

Course Title : AI Assisted Coding Course

Code : 23CS002PC304

Semester : VI

Academic Session : 2025-2026

Name of Student : KARTHISHA

Enrollment No. : 2303A52099

Batch No. : 33

Task 1: Fairness in AI Code Generation

Prompt:

Provide AI with tasks involving different user groups (such as age and gender) and generate outputs. Compare the outputs to evaluate fairness and identify any bias in AI-generated results.

Code:

```
# Simulated AI hiring decision system

def evaluate_candidate(age, gender, skills):
    if "Python" in skills and "ML" in skills:
        if age < 50:   # Biased condition
            return "Selected for interview"
        else:
            return "Rejected due to age"
    else:
        return "Rejected due to skills"

# Test cases
candidates = [
    {"name": "Male Candidate", "age": 25, "gender": "Male", "skills": ["Python", "ML"]},
    {"name": "Female Candidate", "age": 25, "gender": "Female", "skills": ["Python", "ML"]},
    {"name": "Senior Candidate", "age": 55, "gender": "Male", "skills": ["Python", "ML"]}
]

for c in candidates:
    result = evaluate_candidate(c["age"], c["gender"], c["skills"])
    print(c["name"], "→", result)
```

Sample Input/Sample:

```
Male Candidate → Selected for interview
Female Candidate → Selected for interview
Senior Candidate → Rejected due to age
```

Code Explanation:

1. This program simulates an AI-based hiring system.
2. Candidates are evaluated based on age and skills.
3. A biased condition rejects candidates above a certain age.
4. Even with identical skills, outcomes differ unfairly.
5. This demonstrates how bias can exist in AI systems.
6. Such bias can lead to discrimination in hiring processes.
7. AI models must be audited and trained on fair datasets.
8. Ethical AI design ensures equal opportunity for all users.

Task 2: Transparency of AI Decisions

Prompt:

Generate code using AI for a specific task and analyze how transparent the AI-generated logic is. Evaluate whether the decision-making process is clear and understandable to users.

Code:

```
# AI-generated loan approval system

def loan_approval(income, credit_score):
    if income > 50000 and credit_score > 700:
        return "Loan Approved"
    elif income > 30000 and credit_score > 650:
        return "Loan Approved with Conditions"
    else:
        return "Loan Rejected"

# Test cases
print(loan_approval(60000, 720))
print(loan_approval(35000, 660))
print(loan_approval(20000, 600))
```

Sample Input/Sample:

```
Income: 60000
Income: 35000
Income: 20000
```

```
Loan Approved
Loan Approved with Conditions
Loan Rejected
```

Code Explanation:

1. This task evaluates transparency in AI-generated code.
2. The program makes decisions based on income and credit score.
3. The logic is rule-based and easy to understand.
4. Users can clearly see how decisions are made.
5. However, the reasoning behind threshold values is not explained.
6. In real AI systems, models can be complex and opaque.
7. Lack of transparency can reduce trust in AI decisions.
8. Explainable AI techniques are important for clarity and accountability.

Task 3: Data Misuse and Ethical Risks

Prompt:

Generate AI-based code for a decision-making system and analyze who is responsible for the outcomes. Evaluate accountability issues when incorrect or harmful decisions are produced.

Code:

```
# AI-based decision system (Scholarship Eligibility)

def scholarship_eligibility(marks, income):
    if marks > 85 and income < 300000:
        return "Scholarship Approved"
    elif marks > 70:
        return "Partial Scholarship"
    else:
        return "Not Eligible"

# Test cases
print(scholarship_eligibility(90, 200000))
print(scholarship_eligibility(75, 400000))
print(scholarship_eligibility(60, 100000))
```

Sample Input/Sample:

```
Scholarship Approved
Partial Scholarship
Not Eligible
```

Code Explanation:

1. This task focuses on accountability in AI-generated systems.
2. The program decides scholarship eligibility based on marks and income.
3. While the logic appears simple, errors can still occur.
4. If a deserving student is rejected, responsibility becomes unclear.
5. Accountability may lie with developers, organizations, or AI tools.
6. Lack of clear responsibility can lead to ethical and legal issues.
7. Proper validation and human oversight are necessary.
8. Responsible AI ensures accountability for all decisions made.

Task 4: AI Hallucination in Coding

Prompt:

Generate AI-based code that processes user data and analyze potential privacy risks. Evaluate how sensitive information is handled and suggest improvements for data protection.

Code:

```
# AI-generated Binary Search (Incorrect Version)

def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low < high:   # ❌ ERROR HERE
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Test
arr = [1, 3, 5, 7, 9]
print(binary_search(arr, 9))
```

Correct Code (Fixed Version)

```
# Correct Binary Search

def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:   # ✅ FIXED
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Test
arr = [1, 3, 5, 7, 9]
print(binary_search(arr, 9))
```

Sample Input/Sample:

(AI-Generated — WITH ERROR

-1

Correct Code (Fixed Version)

4

Code Explanation:

1. This task demonstrates AI hallucination in coding.
2. AI-generated code may appear correct but contain logical errors.
3. In this example, Binary Search uses an incorrect loop condition.
4. This causes failure when searching the last element.
5. Such errors are difficult to detect without proper testing.

6. The corrected version fixes the loop condition.
7. Developers must always verify AI-generated code.
8. Testing and validation are essential to avoid hidden bugs.

Task 5: Responsibility in AI Deployment

Prompt:

Develop a Python program simulating an AI-based loan approval system in a banking environment.
Analyze risks, accountability, and propose ethical safeguards for responsible deployment.

Code:

```
# AI-based Loan Approval System

def loan_decision(income, credit_score):
    # Basic AI logic
    if income > 50000 and credit_score > 700:
        return "Loan Approved"
    elif income > 30000 and credit_score > 650:
        return "Approved with Conditions"
    else:
        return "Loan Rejected"

# Test multiple customers
customers = [
    {"name": "Customer A", "income": 60000, "credit_score": 750},
    {"name": "Customer B", "income": 40000, "credit_score": 660},
    {"name": "Customer C", "income": 25000, "credit_score": 600}
]

# Output results
for c in customers:
    result = loan_decision(c["income"], c["credit_score"])
    print(c["name"], "→", result)
```

Sample Input/Sample:

```
Customer A → Loan Approved
Customer B → Approved with Conditions
Customer C → Loan Rejected
```

Code Explanation:

1. This program simulates an AI-based banking decision system.
2. It evaluates customers using income and credit score.
3. The logic determines loan approval or rejection.
4. However, fixed conditions may lead to unfair decisions.

5. AI systems can impact users financially, making risks critical.
6. Accountability is shared between developers and organizations.
7. Ethical practices are required to ensure fairness and safety.
8. Responsible AI deployment improves trust in banking systems.

Task 6: Ethical Issues in Automation

Prompt:

Develop a Python program that simulates an AI-based hiring filter system to automate candidate selection. Analyze fairness, identify ethical concerns, and suggest improvements.

Code:

```python
# AI-based Hiring Filter System

def hiring_decision(age, experience, skills):
    # Automated filtering logic
    if "Python" in skills and experience >= 2:
        if age < 40:   # ⚠️ Potential bias
            return "Selected"
        else:
            return "Rejected due to age"
    else:
        return "Rejected due to skills/experience"

# Candidates data
candidates = [
    {"name": "Candidate A", "age": 25, "experience": 3, "skills": ["Python", "ML"]},
    {"name": "Candidate B", "age": 42, "experience": 5, "skills": ["Python", "ML"]},
    {"name": "Candidate C", "age": 30, "experience": 1, "skills": ["Python"]}
]

# Output results
for c in candidates:
    result = hiring_decision(c["age"], c["experience"], c["skills"])
    print(c["name"], "→", result)
```

Improved Code (Fair Version)

```python
# Improved Hiring System (Fair Approach)

def hiring_decision(age, experience, skills):
    if "Python" in skills and experience >= 2:
        return "Selected"
    else:
        return "Rejected based on skills/experience"

# Same candidates
candidates = [
    {"name": "Candidate A", "age": 25, "experience": 3, "skills": ["Python", "ML"]},
    {"name": "Candidate B", "age": 42, "experience": 5, "skills": ["Python", "ML"]},
    {"name": "Candidate C", "age": 30, "experience": 1, "skills": ["Python"]}
]

for c in candidates:
    print(c["name"], "→", hiring_decision(c["age"], c["experience"], c["skills"]))
```

Sample Input/Sample:

```
Candidate A → Selected
Candidate B → Rejected due to age
Candidate C → Rejected due to skills/experience
```

Code Explanation:

1. This task demonstrates ethical issues in AI automation systems.
2. The initial hiring filter uses biased logic based on age.
3. This leads to unfair rejection of qualified candidates.
4. Such bias can cause discrimination in real-world hiring.
5. Ethical concerns include fairness, transparency, and accountability.
6. The improved system removes biased conditions.
7. Decisions are now based only on relevant factors like skills and experience.
8. Ethical AI design ensures fair and unbiased automation.

Task 7: AI and Intellectual Property

Prompt:

Generate code using AI for a well-known problem (e.g., Fibonacci sequence). Compare it with existing solutions and analyze plagiarism and copyright concerns.

Code:

```
# Fibonacci using recursion (AI-generated)

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# Print first 6 terms
for i in range(6):
    print(fibonacci(i), end=" ")
```

Sample Input/Sample:

```
0 1 1 2 3 5
```

Code Explanation:

1. This task examines intellectual property issues in AI-generated code.
2. The Fibonacci program is a well-known problem with standard solutions.
3. AI-generated code often resembles existing implementations.
4. This creates a risk of plagiarism and lack of originality.
5. It becomes difficult to determine ownership of the code.
6. Students must ensure proper usage and avoid copying blindly.
7. Ethical practices include modifying and understanding the code.
8. Responsible use of AI helps maintain academic integrity.

Plagiarism s Copyright Concerns

AI may generate code similar to existing sources
Risk of unintentional plagiarism
Difficult to identify original author
Using such code without attribution may violate academic rules